CLOS CALCULATIONS

DÆDÆLUS Research *August 27, 2025*

RESEARCH

CLOS CALCULATIONS

Clos networks approximate a high-radix switch using identical network elements. The basic formula is : $(5k^2/4)$ k-port switches support $(k^3/4)$ hosts.

Examples:

12-ports: 432 hosts using 180 switches (41.7%)
24-ports: 3456 hosts using 720 switches (20.8%)
48-ports: 27,648 hosts using 2,880 (10.4%)
96-ports: 221164 hosts using 11,520 (5.2%)

For small # hundreds of hosts, the overhead of the switch boxes is high. Higher radix switches are also disproportionately higher in cost, and require longer (more expensive) cables. The above numbers are for only *one* slice of a Clos – one port per server. A minimum of two slices is needed for (1+1) redundancy, and frequently, up to *four* slices are used to provide the required redundancy and bandwidth for a modern data-center. The most typical configuration is 48-port TOR switches, which supports up to 40 servers/rack.

HIGH RADIX SWITCHES

Security: the uptake of SDN is proof of the problem. It means there is no solution. If there are many different 'solutions' to a problem' it means there is no solution that yet works.

We fail over automatically without human intervention, unless the network is partitioned, and as long as we have at least one link left. When a high-radix switch fails, there is a much greater risk that the network will partition. As far as high-performance distributed systems are concerned, a clogged network (delayed packets), or even a network that is not clogged but capriciously drops, duplicates, and reorders packets is indistinguishable from a partition.

Directly connected networks have much greater partition resiliency, and lower cost.

The argument for needing separate hardware (ASICs) to switch the packet belongs to the shared (monolithic) mindset. The more entities that have to share something, the more costly it becomes, and the more complexity is required to make sure that sharing is reliable and fair to all participants.

There is one argument, however, for continuing to use TOR switches into the Clos, and that is because this is where the interface from the

old world can take place. Although our TRAPH technologies are applicable to managing an entire datacenter infrastructure, there are good reasons to have different roles have different vantage points. Infrastructure managers need oversight and control over inter-tenant relationships, and developers need control over the their intra-tenant relationships. In today's infrastructures, these two very different roles are conflated, resulting in the major complexities, delays, and error-prone nature of network management.

We can overcome this in the market by using the TRAPH technology on switches to manage *inter*-tenant relationships, from the vantage point required by an infrastructure manager; and also using the same TRAPH technology on cells (servers) to manage *intra*-tenant relationships, i.e. the application graph built from sets of microservices.

DATACENTER NETWORK

The current trend in datacenter networks aims to approximate one big L2 non-blocking switch by combining 100's of smaller switches in a Clos or Leaf-Spine topology. The premises behind this trend include:

- Any server anywhere
- Any router anywhere
- Any appliance anywheere
- Many VM anywhere
  - Any IP address anywhere
  - Any subnet anywhere
- Any storage anywhere
- Assumes Minimal congestion issues.
- Total flexibility for power use
- Clos Design Goals
  - Full bisection bandwidth between all pairs of hosts
    Aggregate bandwidth = # hosts × host NIC capacity

FUNGIBILITY VS. COUPLING

The instinct to make this resource (datacenter network bandwidth) 'fungible' is well intended. However, a single mainframe is also fungible. Trying to scale out the high radix switch abstraction with commodity switches [Vadhat] is also well intended, after all – scale out has proven to be extraordinary successful when it comes to servers. However, whereas

scale-out servers are either loosely coupled or not coupled at all, commodity switches cooperating to be a single high-radix switch are extremely *tightly* coupled; its not the same at all as scaling out the servers.

Our thesis is that the tightly coupled nature of the high-radix switch illusion will break down, just like all other monolithic elements in the datacenter have broken down due to layers and layers of complexity piled on top.

It won't take much longer before, this will be seen (just like SAN's) as a failed architectural theory; but which caused so much pain and complexity that they must eventually be abandoned.

This is a major market opportunity. Whoever emerges as the thought leader (and has the github community following to prove it), will dominate the next generation datacenter infrastructures.

All the above arguments are about the conflict between the old world of network engineering and the new world of infrastructure topology engineering. Our arguments challenge the prevailing wisdom along traditional cost, scalability, performance, and manageability dimensions. While there is clear evidence for superiority of DCN's over the shared monolithic network in many of these areas, there is one area where DCN's win hands down, every time, and that is in their ability to provide securely segmented resources that guarantee privacy. When we don't have to put all our packets through a centralized monolithic packet sniffer, then any vulnerability or backdoor in those packet sniffers will be unable to compromise the privacy of that data, or represent an adversary in thwarting communications between microservices that have private channels between them.

VALUE PROPOSITION: DEFENSE AGAINST MISCONFIGURATIONS

Configuration errors are a major cause of system failures and vulnerabilities. Many configuration issues manifest themselves in ways similar to software bugs such as crashes, hangs, silent failures. It leaves users clueless and forced to report to developers for technical support, wasting not only users' but also developers? precious time and effort. Unfortunately, unlike software bugs, many software developers take a much less active, responsible role in handling configuration errors because "they are users' faults."

Configuration errors are a major cause of system failures. Barroso and Hölzle report that misconfigurations are the second major cause of service-level failures at one of Google's services. Similar findings are reported by other companies: Every year, sometimes several times a year, Amazon EC2, Microsoft Azure and Facebook, experienced a number of misconfiguration-induced outages that affect millions of their customers.

MONOLITHIC

Networks are the last vestige of *monolithic* scale-up systems thinking. After experiencing the pain of ever-more complexified monolithic computers, Operating Systems and Applications limit the scalability and manageability of our datacenters, we are seeing revolutions occur because of their deconstruction. Scale-out commodity hardware, containers, and now stripped-down, modular operating systems.

CLUSTER SIZE DISCUSSION

One of the largest Cassandra production deployments is Apple's, ∼ 75,000 nodes storing > 10 PB of data. Other large installations include Netflix (2,500 nodes, 420 TB, ∼ 1 trillion requests per day), Chinese search engine Easou (270 nodes, 300 TB, ∼ 800 million requests per day), and eBay (over 100 nodes, 250 TB).

However, these numbers say nothing about the maximum cluster size for each Cassandra ring. Datastax's CTO, Jonathan Ellis says the largest he has seen is ∼ 400 nodes.

This is typical of many datacenter / web-services applications. Individual applications most often have 10's to 100's of microservices under a single management domain in their neighborhood; *not* the 1000's or 10,000's nodes in the whole company may have.

We must also not forget that when we package microservices in VM's or containers, and many VM's or containers per node, we can aggregate many of these paths within the same TRAPH. This is why TRAPHs are stacked trees, with each level able to inherit the properties of the topologies below them, and build new (subset) properties and topologies above them.

SHORTEST PATH BRIDGING

An alternative architecture to approximate a high radix bridge is the IEEE Shortest Path Bridging (SPB) Protocol.

802.1aq Shortest Path Bridging allows for true shortest path routing, multiple equal cost paths, much larger layer 2 topologies, faster convergence, vastly improved use of the mesh topology, single point provisioning for logical membership (E-LINE/E-LAN/E-TREE etc), abstraction of attached device MAC addresses from the transit devices, head end and/or transit multicast replication.

Applications consist of STP replacement, Data Center L2 fabric control, L2 networks that scale to 1000 bridges:

- Use of arbitrary mesh topologies.
- Use of (multiple) shortest paths.

- Efficient broadcast/multicast routing and replication points.
- Avoid address learning by tandem devices.
- Get recovery times into 100's of millisecond range for larger topologies.
- Good scaling without loops.
- Allow creation of very many logical L2 topologies (subnets) of arbitrary span.
- Maintain all L2 properties within the logical L2 topologies (transparency, ordering, symmetry, congruence, shortest path etc).
- IEEE protocol builds on 802.1 standards
- A new control plane for Q-in-Q and M-in-M
- Leverage existing inexpensive ASICs
- Q-in-Q mode called SPBV
- M-in-M mode called SPBM
- Backward compatible to 802.1
- 802.1ag, Y.1731, Data Center Bridging suite
- Multiple loop free shortest paths routing
- Excellent use of mesh connectivity
- Currently 16, path to 1000's including hashed per hop.
- Optimum multicast
- head end or tandem replication

Light weight form of traffic engineering:

- Head end assignment of traffic to 16 shortest paths.
- Deterministic routing - offline tools predict exact routes.
- Scales to 1000 or so devices
- Uses IS-IS already proven well beyond 1000.
- Huge improvement over the STP scales.
- Good convergence with minimal fuss
- sub second (modern processor, well designed)
- below 100ms (use of hardware multicast for updates)

- Includes multicast flow when replication point dies. Pre-standard seeing 300ms recovery @ 50 nodes.
- IS-IS
- Operate as independent IS-IS instance, or within IS-IS/ IP, supports Multi Topology to allow multiple instances efficiently.

  Membership advertised in same protocol as topology:

- Minimizes complexity, near plug-and-play
- Support E-LINE/E-LAN/E-TREE
- All just variations on membership attributes.
- Address learning restricted to edge (M-in-M)
- FDB is computed and populated just like a router.
- Unicast and Multicast handled at same time.
- Nodal or Card/Port addressing for dual homing.
- Computations guarantee ucast/mcast...
- Symmetry (same in both directions)
- Congruence (unicast/multicast follow same route)
- Tune-ability (currently 16 equal costs paths – opaque allows more)